From File Labels to Byte Evidence: Budget-Aware Weakly Supervised Localization in Static PE Binaries

UA-SAHI-Mal — Venue-neutral pre-results manuscript draft

[Author 1] [Author 2]

Draft: 2026-09-04

Abstract

Static malware detectors can classify a Portable Executable (PE) file without identifying the byte regions or functions that justify the decision. This gap limits analyst trust and leaves reverse engineers with most of the original program to inspect. Building supervised localizers is difficult because public malware corpora predominantly provide file-level labels, while independently validated component-level annotations remain scarce, heterogeneous, or unavailable at scale. We present UA-SAHI-Mal, a budget-aware weakly supervised framework that learns from file labels to rank and recursively refine security-relevant byte intervals in statically analyzable PE binaries. The framework introduces PEAtlas, an invertible, structure-aware mapping between file offsets, PE sections, virtual addresses, program-analysis entities, and image coordinates. A relation-aware multiple-instance learner produces a coarse evidence map; a structure-guided feature upsampler reconstructs the map at higher resolution; and a guarded subset selector allocates a fixed analysis budget to high-value and uncertain regions. The final output is a ranked union of byte intervals, with masks and bounding boxes derived only for visualization and detector compatibility. To avoid evaluating localization solely against model explanations or a single rule engine, we define a non-pooled, multi-tier benchmark comprising controlled-provenance builds, rule-derived silver evidence, source-grounded analyst-audited gold annotations, and an in-the-wild analyst study. This pre-results manuscript specifies the threat model, hypotheses, baselines, leakage controls, statistical analysis, and reporting protocol. Experimental values are intentionally left as [TBD] until measured.

Contents

1. Introduction

Static machine-learning detectors have made it possible to score Windows Portable Executable (PE) files at scale using engineered features, raw bytes, or image-like representations (Anderson and Roth 2018; Raff et al. 2018; Nataraj et al. 2011). Their standard output, however, is a file-level label or probability. A malware analyst who receives a positive verdict must still identify the functions, basic blocks, strings, embedded objects, or byte regions that implement the behavior of interest. This distinction is operationally important: a high-confidence classifier can reduce triage volume while providing little guidance for reverse engineering.

Prior work has begun to address component localization. DeepReflect ranks suspicious functions and basic blocks without requiring component labels for training and showed that such prioritization can reduce analyst workload (Downing et al. 2021). Capability systems such as capa identify code-level behaviors through expert-authored rules (Mandiant FLARE Team 2026). More recent PE models use multiple-instance learning (MIL) or position-preserving features to expose influential byte segments (Peters and Farhat 2023; Kim, Trung, and Pham 2026; Yevsikov and Nissim 2026). These systems establish that local evidence is useful, but they do not yield a widely adopted public benchmark in which raw PE bytes, benign and malicious files, independently validated malicious components, reversible address mappings, and leakage-controlled splits are all available together.

The absence of such a benchmark should not be interpreted as proof that no component-level data exist, nor does it make supervised localization impossible. Private corpora, manual reverse engineering, active learning, and source-grounded builds are possible. The practical problem is narrower: public, reproducible evaluation of malicious-component localization remains constrained by annotation cost, licensing, malware-handling restrictions, disassembly uncertainty, and ambiguity about what constitutes a malicious component.

At the same time, weak supervision does not solve localization automatically. A positive malware file contains substantial benign or shared code, and a behavior may arise through interactions among multiple functions. Attention weights or class activation maps can reveal features used by a classifier, but they are not, by

themselves, evidence that a highlighted region implements malicious behavior. High-resolution patch selection also has substantial precedent in MIL and computational pathology (Ilse, Tomczak, and Welling 2018; Lu et al. 2021; Li, Li, and Eliceiri 2021; Shao et al. 2021), and high-resolution malware-image MIL has already used attention to prioritize patches (Peters and Farhat 2023). Consequently, a contribution based only on "CAM or MIL followed by top-K tiles" would be insufficient.

This paper studies a more specific problem: **under an explicit analysis budget, can a model trained primarily with file-level labels retrieve byte intervals that independent evidence and analysts associate with security-relevant malware components?** We introduce UA-SAHI-Mal, a hierarchical evidence-retrieval framework for statically analyzable PE files. The system first constructs an invertible, structure-aware representation, PEAtlas. A relation-aware MIL model learns a coarse tile-level evidence distribution. A structure-guided instance of Upsample Anything (UA) (Seo, Hamilton, and Kim 2026) reconstructs higher-resolution evidence, and a guarded budget selector balances high scores, uncertainty, and section coverage. Selected regions are recursively refined until the budget is exhausted or the desired byte granularity is reached.

The canonical output is a ranked set of byte intervals and mapped program-analysis entities, not a two-dimensional bounding box. Bounding boxes are derived views because a contiguous one-dimensional byte interval can wrap across image rows, and optimized functions can occupy multiple disjoint code chunks. This design keeps evaluation in the native PE coordinate system while retaining compatibility with image models and visual analyst interfaces.

We further introduce an evaluation protocol that separates evidence by provenance rather than pooling incompatible labels. Controlled-provenance builds test mapping and shortcut resistance. YARA and capa matches provide scalable silver evidence. Source-grounded builds and selected DeepReflect artifacts form an analyst-audited gold set. Finally, a blinded analyst study evaluates whether ranked regions reduce time and code reviewed. Each tier is reported separately.

This work makes the following intended contributions, subject to the empirical gates defined in Section 8:

1. **A reproducible audit and task definition.** We formalize public static-PE localization requirements, distinguish file detection from component retrieval, and document inclusion and exclusion criteria for existing datasets and methods.
2. **PEAtlas.** We provide an invertible mapping among raw file offsets, PE sections, RVAs, disassembly entities, and image coordinates, preserving disjoint byte intervals as the canonical annotation.
3. **A budget-aware weak localizer.** UA-SAHI-Mal combines relation-aware MIL, structure-guided evidence reconstruction, counterfactual and layout-consistency objectives, and guarded hierarchical selection under an explicit cost constraint.
4. **A non-pooled multi-tier benchmark.** We construct controlled-provenance, silver, source-grounded gold, and analyst-evaluation tiers with explicit provenance and leakage controls.
5. **Analyst-centered evaluation.** We report evidence recall at fixed byte, function, latency, and compute budgets, along with time-to-first-relevant-function and reviewed-code reduction.

No numerical result is claimed in this draft. All values, sample counts, and effect sizes marked `[TBD]` must be replaced only after executing the registered protocol.

2. Background, Motivation, and Scope

2.1 Static PE malware detection

A PE file consists of headers, sections, optional overlays, and metadata that define how on-disk bytes are mapped into memory. Large malware datasets such as EMBER and EMBER2024 support file-level learning from static features (Anderson and Roth 2018; Joyce et al. 2025). BODMAS provides temporal labels and disarmed malware binaries, while its original benign binaries are not publicly distributed (Yang et al. 2021). These resources are suitable for detection benchmarks but do not directly provide malicious-component intervals.

Raw-byte neural networks avoid manual feature engineering, while visualization methods reshape bytes into grayscale or multichannel images (Raff et al. 2018; Nataraj et al. 2011). Image representations enable the use of convolutional or vision-transformer backbones, but they introduce layout choices. Adjacent pixels at a row boundary may be far apart in program structure, and visual edges do not necessarily correspond to semantic function boundaries. Our evaluation therefore compares one-dimensional raw-byte, fixed-width raster, and section-aligned representations.

2.2 Local evidence and analyst workload

DeepReflect demonstrated that function prioritization can reduce reverse-engineering effort and that unsupervised reconstruction can identify malware components without component labels during training (Downing et al. 2021). capa complements learned methods with expert-authored capability rules operating at instruction, basic-block, function, and file scopes (Mandiant FLARE Team 2026). YARA provides exact offsets and lengths for matched strings or byte patterns (VirusTotal 2026). These tools are useful baselines and annotation sources, but their outputs have different semantics:

- a YARA match is a signature interval;
- a capa match is a capability-bearing program-analysis scope;
- a model attribution is a region influential to a prediction;
- a source-grounded annotation is a component linked to known implementation and analyst judgment.

We retain these distinctions throughout the paper.

2.3 Weakly supervised localization

In MIL, a bag is labeled while individual instances are unlabelled (Ilse, Tomczak, and Welling 2018). Attention-based MIL can assign interpretable weights to instances, and pathology systems such as CLAM, DSMIL, and TransMIL have localized diagnostically relevant regions in gigapixel images using slide-level labels (Lu et al. 2021; Li, Li, and Eliceiri 2021; Shao et al. 2021). Malware files create a harder identification problem: positive bags may contain shared libraries, runtime code, benign resources, and multiple cooperating malicious functions. A file-level label does not imply that every tile is positive, or even that one isolated tile is sufficient.

UA-SAHI-Mal treats benign bags as stronger negative supervision and positive bags as positive-unlabeled mixtures. It uses cross-layout consistency and counterfactual tests to reduce dependence on arbitrary image placement. Even with these constraints, the output remains a ranked hypothesis until validated against independent evidence.

2.4 Slicing and feature upsampling

SAHI applies an object detector to overlapping image slices and merges detections, primarily to improve small-object detection (Akyon, Altinuc, and Temizel 2022). Our system is inspired by its resolution-preserving slicing principle but differs in two ways. First, PE localization begins without dense object labels. Second, the primary output is byte evidence rather than natural-image objects. We therefore use the term **budgeted hierarchical slicing** for the core retrieval mechanism. The name "SAHI" is retained in the system name to identify its design lineage; any experiment described as "SAHI" uses an actual detector or segmenter on slices and follows the original merge protocol.

Upsample Anything learns a per-input anisotropic Gaussian operator to transfer low-resolution features or probability maps to high resolution (Seo, Hamilton, and Kim 2026). Its edge-aware reconstruction is potentially useful, but raw-byte edges need not be semantic. We evaluate byte-only and structure-guided variants and charge all test-time optimization cost to the end-to-end budget.

3. Threat Model and Task Definition

3.1 Scope

The primary scope is statically analyzable native Windows PE32 and PE32+ files. The system does not execute samples. The main benchmark excludes, or separately labels:

- packed samples that cannot be unpacked with the declared preprocessing pipeline;
- self-modifying code;
- managed .NET assemblies;
- kernel drivers requiring specialized semantics;
- files for which the selected disassembler cannot recover stable function boundaries.

Packed and optimized files are evaluated as domain-shift strata where feasible, not silently mixed into the primary gold set.

3.2 Adversary

The adversary may alter nonsemantic file layout, add padding or overlays, reorder sections when functionality permits, use common packers, or introduce benign-looking code. The initial study does not claim robustness to an adaptive attacker with full knowledge of the model and arbitrary semantics-preserving binary rewriting. We nevertheless include padding, row-width, section-layout, compiler, optimization, and selected packing perturbations to detect representation shortcuts.

3.3 Input and output

For a PE file F with on-disk byte sequence

$$\mathbf{b} = (b_0, b_1, \ldots, b_{N-1}), \qquad b_o \in \{0, \ldots, 255\},$$

UA-SAHI-Mal receives the bytes and statically derived structural metadata. It returns a ranked collection

$$\mathcal{R}(F) = \{(I_k, s_k, m_k)\}_{k=1}^{K},$$

where I_k is a union of one or more half-open file-offset intervals, s_k is a confidence score, and m_k contains section, RVA, mapped function/basic-block identifiers, and provenance metadata. The total cost must satisfy

$$C(\mathcal{R}) \leq B,$$

where B may be defined in bytes, tiles, functions, encoder calls, FLOPs, or wall-clock time.

3.4 Target semantics

The target is **security-relevant evidence associated with malicious behavior**, not every byte in a malware file. For gold annotations, analysts assign one of:

- `MALICIOUS_COMPONENT`: directly implements or enables the malicious objective;
- `SUPPORTING_COMPONENT`: infrastructure required by the malicious component but not independently malicious;
- `BENIGN_OR_SHARED`: runtime, library, or ordinary functionality;
- `UNCERTAIN`: insufficient evidence for adjudication.

Primary gold localization metrics use `MALICIOUS_COMPONENT`; sensitivity analyses may include supporting components. Silver evidence is never relabeled as gold without analyst adjudication.

4. Auditing the Ground-Truth Gap

4.1 Audit question

We ask whether a publicly accessible benchmark satisfies all of the following:

1. raw PE bytes or a provably reversible representation;
2. benign and malicious files;
3. independently validated malicious-component labels;
4. a mapping from labels to file offsets and program-analysis entities;
5. leakage-controlled family, temporal, source, and duplicate splits.

The audit protocol, search strings, databases, dates, inclusion criteria, and exclusion reasons will be released in Appendix A and the artifact repository. We use the cautious conclusion "no widely adopted public benchmark was identified within the audit scope," rather than a universal nonexistence claim.

4.2 Position of existing resources

Resource	Raw/reversible PE	Benign + malicious	Local evidence	Evidence status	Intended use
EMBER / EMBER2024	Features and metadata; raw files not generally included	Yes	No component intervals	File labels	Detection and temporal baselines
BODMAS	Disarmed malware binaries; benign originals not public	Labels include both	No component intervals	File labels	Detection and time split
DeepReflect artifacts	Limited malware/source artifacts	Limited	Function/basic-block evidence	Source/analyst-grounded seed	Gold seed and localization baseline
EMBER2024-capa	Function bytes and disassembly for many malicious functions	No balanced raw-file corpus	Capability-bearing functions	Silver	Pretraining and silver evaluation
YARA	Applied to local corpus	Depends on corpus	Exact match offsets	Silver signature	Rule agreement
capa	Applied to local corpus	Depends on corpus	Instruction/BB/function capabilities	Silver capability	Rule agreement and baseline
Source-compiled corpus	Yes	Constructed controls	Source-linked interval unions	Gold after audit	Primary controlled gold

The final paper will expand this table with citations and dataset version identifiers.

4.3 Why supervised detection remains a baseline

The lack of a large public gold corpus does not make supervised learning invalid. Once Tier 3 annotations exist, we train supervised detector/segmenter baselines on the available gold training portion, use cross-validation where necessary, and report uncertainty. This comparison prevents weak supervision from receiving an artificial advantage.

5. PEAtlas: Invertible Structure-Preserving Representation

5.1 Coordinate spaces

PEAtlas distinguishes five coordinate systems:

1. file offsets on disk;
2. RVAs in the loaded image;
3. VAs after adding the image base;
4. instruction, basic-block, and function entities recovered by analysis;
5. pixels or patches in a model representation.

A PE section records, among other fields, `VirtualAddress`, `VirtualSize`, `PointerToRawData`, and `SizeOfRawData`; raw and virtual extents may differ (Microsoft 2025b). Therefore, every conversion can fail explicitly:

$$\text{rva2off}(r) \in \{o, \perp_{\text{unmapped}}\},$$

$$\text{off2rva}(o) \in \{r, \perp_{\text{nonloaded}}\}.$$

Certificate data, debug data, overlays, and zero-filled virtual ranges are tagged rather than coerced into a false one-to-one mapping.

5.2 Fixed-width raster

For a baseline image width W, a valid file offset o maps to

$$\pi_W(o) = (x, y) = (o \bmod W, \lfloor o/W \rfloor).$$

The inverse is $o = yW + x$ for valid pixels. Each pixel retains its original offset. A byte interval crossing a row boundary becomes multiple horizontal runs, not one enclosing rectangle.

5.3 Section-aligned atlas

The proposed layout assigns each section and nonsection region to a distinct band. Alignment pixels map to $\perp$ and are excluded from loss and metrics. The tensor includes:

$$X = [v, e, q, r, w, x, m_{raw}, m_{loaded}],$$

where v is normalized byte value, e local entropy, q a section embedding or ID channel, $r/w/x$ are section permissions, and masks identify valid raw and memory-mapped bytes.

The atlas stores an explicit pixel-to-offset lookup table and an interval-to-mask routine. We release round-trip tests for at least `[TBD: 10^6 or more]` randomly sampled offsets and edge cases.

5.4 Program-analysis mapping

A disassembler returns instruction, basic-block, and function address sets. PEAtlas maps each recovered address to a set of file-offset intervals when possible. A function may map to disjoint intervals because of inlining, cold-code splitting, or compiler/linker optimization. Optimized code can eliminate, merge, or inline functions, and identical COMDAT folding can merge code (Microsoft 2021, 2025a). Consequently, the annotation schema stores interval unions rather than a single start/end pair.

6. UA-SAHI-Mal

6.1 Overview

UA-SAHI-Mal comprises five stages:

1. PEAtlas construction;
2. coarse tile encoding and relation-aware MIL;
3. structure-guided evidence reconstruction;
4. guarded subset selection under budget;
5. recursive fine localization and evidence export.

A system diagram will appear as Figure 1.

6.2 Tile encoder and relation-aware MIL

PEAtlas is divided into possibly overlapping coarse tiles t_j with positional and section metadata u_j. An encoder produces

$$h_j = f_\theta(t_j).$$

A relation-aware aggregator, instantiated with ABMIL, DSMIL, or a transformer during ablation, computes context-dependent evidence scores

$$a_j = g_\phi(h_j, u_j, \{h_\ell, u_\ell\}_{\ell \neq j})$$

and a file representation

$$z = \sum_j \mathrm{softmax}(a)_j h_j.$$

The file probability is $p = \sigma(q(z))$.

For a benign bag, all tiles receive negative-instance regularization. For a malicious bag, tile labels are unobserved and are treated as positive-unlabeled. The model is not allowed to infer gold labels from YARA or capa during the file-label-only training condition.

6.3 Coarse evidence map

Tile scores are projected into a low-resolution evidence map E_{lr}. Overlap is combined using a calibrated aggregation rule. We compare mean, maximum, noisy-OR, and learned aggregation. Scores are temperature-calibrated on validation data without gold test annotations.

6.4 Structure-guided UA

We optimize a per-file upsampling operator using PEAtlas guidance G:

$$\phi^* = \arg\min_\phi \mathcal{L}_{guide}(U_\phi(G_{lr}, G_{hr}), G_{hr}),$$

then transfer the operator to evidence:

$$E_{hr} = U_{\phi^*}(E_{lr}, G_{hr}).$$

We test two guidance conditions:

- byte-only grayscale guidance;
- structural guidance using byte, entropy, section, permissions, and validity masks.

Nearest, bilinear, bicubic, guided/JBU, and UA are compared. UA optimization time and memory are included in all online efficiency numbers.

6.5 Guarded budget selector

For candidate tile j, the selector computes

$$s_j = \hat{e}_j + \alpha \hat{u}_j + \beta \hat{n}_j + \gamma \hat{c}_j,$$

where $\hat{e}_j$ is evidence, $\hat{u}_j$ predictive or ensemble uncertainty, $\hat{n}_j$ diversity/novelty relative to selected tiles, and $\hat{c}_j$ coverage value for underrepresented executable sections or spatial regions.

We select S subject to

$$\max_{S \subseteq \mathcal{T}} \sum_{j \in S} s_j, \qquad \text{s.t. } \sum_{j \in S} c_j \leq B.$$

A differentiable soft-selection relaxation may be used during training; inference uses a deterministic algorithm with fixed tie breaking. The paper reports both tile-count and wall-clock budgets.

6.6 Hierarchical refinement

Selected tiles are subdivided and re-encoded at higher resolution. Refinement continues until one of the following holds:

- the minimum interval granularity is reached;
- evidence concentration exceeds threshold τ_e;
- uncertainty falls below τ_u;
- the remaining budget cannot fund another split.

The fine stage produces a score for each retained subinterval. Neighboring intervals may be merged only when the merge does not bridge unrelated invalid or low-score bytes beyond a declared tolerance.

6.7 Training objective

The complete objective is

$$\mathcal{L} = \mathcal{L}_{file} + \lambda_{neg}\mathcal{L}_{neg} + \lambda_{cons}\mathcal{L}_{layout} + \lambda_{cf}\mathcal{L}_{cf} + \lambda_{stab}\mathcal{L}_{stab} + \lambda_{cost}\mathcal{L}_{cost}.$$

File classification

$$\mathcal{L}_{file} = \mathrm{BCE}(p, y).$$

Negative-instance regularization

For benign files,

$$\mathcal{L}_{neg} = \frac{1}{|\mathcal{T}|} \sum_{j} -\log(1 - \sigma(a_j)).$$

Cross-layout consistency

Let T_1 and T_2 be invertible layout transformations, such as different raster widths or raster versus section atlas. After mapping evidence back to offsets,

$$\mathcal{L}_{layout} = D(E^{off}_{T_1}, E^{off}_{T_2}).$$

Counterfactual sufficiency and comprehensiveness

Let F_S retain selected bytes/tiles with a neutralized context, and $F_{\setminus S}$ neutralize selected evidence while preserving the rest. We encourage

$$p(F_S) \approx p(F), \qquad p(F_{\setminus S}) < p(F) - \delta.$$

Controls verify that neutralization does not corrupt PE parsing or introduce an obvious marker.

Stability

Functionality-preserving or representation-preserving perturbations $A(F)$ include permitted padding, row-width changes, and benign overlay modifications:

$$\mathcal{L}_{stab} = D(E^{off}(F), E^{off}(A(F))).$$

Cost

$$\mathcal{L}_{cost} = \max(0, C(S) - B).$$

7. Multi-Tier Evaluation Benchmark

The tiers differ in semantic strength and are never pooled into a single localization score.

7.1 Tier 0: Mapping tests

Tier 0 verifies PEAtlas rather than machine learning. Tests include:

- file-offset/pixel round trips;
- RVA/file-offset conversion for raw, zero-filled, and unmapped ranges;
- section alignment and overlay handling;
- interval-to-mask-to-interval exact recovery;
- agreement among at least two parsers on supported fields;
- disjoint function chunk representation.

A release gate requires zero unexplained mapping errors on `[TBD]` deterministic and randomized cases.

7.2 Tier 1: Controlled-provenance builds

Tier 1 consists of benign host programs linked with safe, source-known modules. Modules emulate security-relevant code patterns without harmful external effects. Examples may include mock process enumeration, local file transformation in a sandbox path, or network-client code redirected to a local test service. Each module has a matched inert control with similar size, entropy, compiler, and placement.

The build system randomizes:

- source module;
- compiler and version;
- architecture;

- optimization mode;
- section placement;
- interval length;
- padding and overlay conditions;
- image layout width.

Ground truth is the union of source-linked code and data intervals produced by debug and map information, verified against disassembly. Tier 1 measures provenance localization and shortcut resistance, not real-world malware accuracy. `secml_malware` manipulations may be used to generate padding/slack perturbations, but the library is not treated as a malicious-payload annotator (Demetrio and Biggio 2021).

7.3 Tier 2: Rule-derived silver evidence

Tier 2 applies YARA and capa to a legally authorized local PE corpus. YARA provides match offsets and lengths; capa provides matched instruction, basic-block, function, or file scopes (VirusTotal 2026; Mandiant FLARE Team 2026). EMBER2024-capa provides function-level bytes and disassembly for a large collection of malicious-file functions and is used for fragment pretraining or silver analysis, not whole-file coordinate reconstruction unless the corresponding authorized PE is available (Joyce and collaborators 2026).

We report:

- overlap with YARA intervals;
- capa function recall@budget;
- capability-stratified results;
- rule-family and capability holdout;
- disagreement between learned evidence and rules.

No silver record is included in gold metrics without manual review.

7.4 Tier 3: Source-grounded analyst-audited gold

Tier 3 combines selected DeepReflect source-grounded artifacts with reproducible source builds. Each project is built in at least two modes:

1. **Gold mode:** optimization and inlining disabled, no identical-code folding, symbols and map data retained;
2. **Shift mode:** realistic optimization enabled to evaluate transfer and annotation fragmentation.

A source-to-binary pipeline yields candidate interval unions. Two analysts independently assign component labels and supporting evidence. Disagreements are adjudicated by a third reviewer or joint review. The released schema records:

```
sample_sha256
project_id
build_id
architecture
compiler_and_flags
packer_status
component_id
component_semantics
label_class
file_offset_intervals
rva_intervals
mapped_functions
source_locations
annotator_ids_pseudonymous
agreement_status
provenance
```

The gold set excludes ambiguous components from primary metrics and reports them separately.

7.5 Tier 4: In-the-wild analyst audit

A blinded crossover study compares UA-SAHI-Mal, DeepReflect, capa, and a no-guidance control. Analysts receive the same time or function budget and may use the same reverse-engineering environment. The presentation order is randomized.

Primary outcomes:

- time to first relevant function;
- number of functions reviewed before first relevant evidence;
- precision among the first K recommendations;
- fraction of analyst-confirmed relevant functions recovered;
- analyst confidence and perceived usefulness.

The study protocol, recruitment criteria, consent, and compensation are reviewed by the applicable institutional process before data collection.

8. Experimental Methodology

8.1 Pre-registration and empirical gates

Before evaluating the test sets, we freeze:

- primary metrics and budgets;
- dataset split hashes;
- gold and silver label versions;
- model-selection metric;
- hyperparameter search space;
- exclusion rules;
- analyst-study endpoints.

The project proceeds through five gates:

- **G0 Mapping integrity:** no unexplained coordinate errors;
- **G1 Gold feasibility:** diverse, analyst-verifiable component intervals can be constructed;
- **G2 Localization signal:** the model exceeds random, entropy, and base MIL rankings on validation gold;
- **G3 Budget Pareto:** the method improves evidence recall at matched cost or reduces cost at matched recall;
- **G4 Analyst utility:** the blinded study shows meaningful workload reduction;
- **G5 Robustness:** evidence is not dominated by layout, padding, or compiler shortcuts.

Failure of G1 changes the claim from malicious-component localization to weak evidence retrieval. Failure of G3 removes the efficiency claim. Failure of G4 limits the paper to a benchmark/method study rather than an analyst-productivity claim.

8.2 Datasets

The final dataset table will report exact counts after legal and technical validation.

Split/source	Role	Files	Families/projects	Gold components	Silver components	Time range
File-label training corpus	Weak training	[TBD]	[TBD]	0	0 or excluded	[TBD]

Split/source	Role	Files	Families/projects	Gold components	Silver components	Time range
Tier 1 controlled provenance	Sanity/robustness	[TBD]	[TBD]	[TBD]	0	generated
Tier 2 rule evidence	Scalable silver	[TBD]	[TBD]	0	[TBD]	[TBD]
Tier 3 source-grounded	Primary gold	[TBD]	[TBD]	[TBD]	optional	[TBD]
Tier 4 analyst study	Utility	[TBD]	[TBD]	analyst-confirmed	optional	[TBD]

EMBER/EMBER2024 features may support file-level baselines, but whole-file image training requires authorized raw PEs. BODMAS disarmed malware binaries may contribute malicious images, while benign raw binaries require a separately licensed corpus (Joyce et al. 2025; Yang et al. 2021).

8.3 Splitting and leakage prevention

We construct grouped splits using:

- chronological collection time;
- malware family;
- source project;
- compiler/toolchain;
- packer;
- exact file hash;
- near-duplicate clustering;
- function hash;
- source-function identity across builds;
- YARA/capa rule and capability group.

All builds of the same source component remain in one split unless the experiment explicitly studies build transfer. EMBER2024-capa function duplicates are removed or grouped before training and evaluation.

8.4 Baselines

File-level detection

- EMBER LightGBM;
- MalConv or MalConv2;
- full-image CNN;
- vision transformer;
- Peters–Farhat high-resolution malware MIL;
- PE-MILCon;
- UA-SAHI-Mal file head.

Localization and ranking

- random-K;
- uniform grid/section-K;
- entropy-K;
- Grad-CAM and a higher-resolution CAM variant;

- Integrated Gradients;
- ABMIL;
- CLAM;
- DSMIL;
- TransMIL;
- Peters–Farhat attention;
- PE-MILCon attention;
- DeepReflect;
- capa and YARA;
- exhaustive tile scan;
- supervised detector/segmenter trained on available Tier 3 gold;
- DLLicious/X-MinHash-style attribution if a faithful implementation is available.

Upsampling

- nearest;
- bilinear;
- bicubic;
- guided/JBU;
- UA byte-only;
- UA structural guidance.

8.5 Metrics

File-level metrics

- PR-AUC;
- ROC-AUC;
- F1;
- TPR at FPR 0.1% and 1%;
- expected calibration error.

Localization metrics

Let G be the union of gold byte intervals and R_B the retrieved intervals under budget B.

Byte recall:

$$\mathrm{Recall}_{byte}(B) = \frac{|R_B \cap G|}{|G|}.$$

Byte precision:

$$\mathrm{Precision}_{byte}(B) = \frac{|R_B \cap G|}{|R_B|}.$$

Function recall:

$$\mathrm{Recall}_{func}(K) = \frac{\#\text{gold functions intersecting top-}K}{\#\text{gold functions}}.$$

We additionally report:

- byte-level AUPRC;
- interval IoU;
- first relevant function rank;

- reviewed-byte fraction;
- reviewed-function fraction;
- derived mask IoU and bbox AP50/AP75 as secondary metrics.

Efficiency metrics

- coarse encoder time;
- UA optimization time;
- selector time;
- refinement time;
- online disassembly/parsing time;
- end-to-end wall-clock latency;
- throughput;
- peak RAM and VRAM;
- encoder/detector calls;
- FLOPs where meaningful.

Preprocessing is reported as online or amortized offline rather than omitted.

8.6 Statistical analysis

We use at least `[TBD]` independent seeds or bootstrap resamples. Confidence intervals are computed at the file or project group level to avoid treating correlated functions as independent. Comparisons use paired bootstrap or permutation tests and report effect sizes. Tier 1, Tier 2, Tier 3, and Tier 4 results are never averaged into one score.

8.7 Research questions

- **RQ1 — File-level validity:** Does weak evidence learning preserve competitive malware detection?
- **RQ2 — Budgeted localization:** At matched cost, does UA-SAHI-Mal retrieve more gold malicious components than attribution, MIL, rules, and DeepReflect?
- **RQ3 — Reconstruction:** Does structure-guided UA improve offset-level localization enough to justify its cost?
- **RQ4 — Objective and guards:** Which constraints reduce false-negative components and representation shortcuts?
- **RQ5 — Analyst utility:** Does ranked evidence reduce time and code reviewed?
- **RQ6 — Robustness:** How does localization change across layout, compiler, optimization, padding, family, temporal, and packing shifts?
- **RQ7 — Component scale:** Conditional on the observed gold-size distribution, are small components disproportionately missed by full-image models?

9. Results

This section is a reporting template. It must not be populated with estimated or illustrative values presented as measurements.

9.1 RQ1: File-level detection

Planned statement: UA-SAHI-Mal achieved `[TBD]` PR-AUC and `[TBD]` TPR at 0.1% FPR on the temporal test set. Relative to `[best baseline]`, the difference was `[TBD]` with a 95% confidence interval of `[TBD]`.

Method	PR-AUC	ROC-AUC	TPR@0.1% FPR	F1	ECE
EMBER LightGBM	TBD	TBD	TBD	TBD	TBD
MalConv	TBD	TBD	TBD	TBD	TBD

Method	PR-AUC	ROC-AUC	TPR@0.1% FPR	F1	ECE
Full-image CNN	TBD	TBD	TBD	TBD	TBD
Peters MIL	TBD	TBD	TBD	TBD	TBD
PE-MILCon	TBD	TBD	TBD	TBD	TBD
UA-SAHI-Mal	TBD	TBD	TBD	TBD	TBD

9.2 RQ2: Gold evidence retrieval

Method	Byte AUPRC	Function Recall@10	Recall@25% byte budget	First-hit rank	Reviewed-byte fraction
Random	TBD	TBD	TBD	TBD	TBD
Entropy	TBD	TBD	TBD	TBD	TBD
Grad-CAM	TBD	TBD	TBD	TBD	TBD
ABMIL	TBD	TBD	TBD	TBD	TBD
DSMIL	TBD	TBD	TBD	TBD	TBD
TransMIL	TBD	TBD	TBD	TBD	TBD
DeepReflect	TBD	TBD	TBD	TBD	TBD
Full scan	TBD	TBD	1.000	TBD	1.000
UA-SAHI-Mal	TBD	TBD	TBD	TBD	TBD

Figure 2 will plot gold component recall against end-to-end latency and reviewed-byte fraction.

9.3 RQ3: Upsampling

Reconstruction	Byte AUPRC	Interval IoU	Layout stability	Added latency	Peak VRAM
Nearest	TBD	TBD	TBD	TBD	TBD
Bilinear	TBD	TBD	TBD	TBD	TBD
Bicubic	TBD	TBD	TBD	TBD	TBD
JBU	TBD	TBD	TBD	TBD	TBD
UA, byte-only	TBD	TBD	TBD	TBD	TBD
UA, structural	TBD	TBD	TBD	TBD	TBD

The text will explicitly report whether UA remains Pareto-efficient after charging its test-time optimization cost.

9.4 RQ4: Ablation

Variant	File PR-AUC	Gold function recall	Byte AUPRC	Latency	Padding stability
Full model	TBD	TBD	TBD	TBD	TBD
– UA	TBD	TBD	TBD	TBD	TBD
– counterfactual loss	TBD	TBD	TBD	TBD	TBD
– layout consistency	TBD	TBD	TBD	TBD	TBD

Variant	File PR-AUC	Gold function recall	Byte AUPRC	Latency	Padding stability
– uncertainty guard	TBD	TBD	TBD	TBD	TBD
– coverage guard	TBD	TBD	TBD	TBD	TBD
byte-only representation	TBD	TBD	TBD	TBD	TBD
raster instead of section atlas	TBD	TBD	TBD	TBD	TBD

9.5 RQ5: Analyst study

Condition	Median time-to-first evidence	Functions reviewed	Precision@10	Confidence	Workload score
No guidance	TBD	TBD	TBD	TBD	TBD
capa	TBD	TBD	TBD	TBD	TBD
DeepReflect	TBD	TBD	TBD	TBD	TBD
UA-SAHI-Mal	TBD	TBD	TBD	TBD	TBD

The analysis will use participant-aware paired statistics and disclose sample size, experience, order effects, and exclusions.

9.6 RQ6: Robustness

We report stratified localization under:

- raster width changes;
- benign padding and overlay insertion;
- compiler and optimization changes;
- family and time shifts;
- packer strata where static analysis succeeds;
- disassembler disagreement.

A model that preserves file classification but moves evidence to inserted padding fails the localization robustness criterion.

9.7 RQ7: Component scale

We first report the empirical distribution of gold interval length, function size, image-mask area, fragmentation, and post-resize pixel count. Only if small components are sufficiently represented do we report scale-stratified recall. COCO small-object thresholds are not adopted without justification because PE byte intervals have different semantics from natural-image objects.

10. Security Analysis

10.1 Shortcut tests

We test whether the model relies on:

- file size;
- section count or names;
- packer markers;
- padding entropy;
- certificate presence;
- import-table artifacts;
- family-specific build signatures;
- image row boundaries.

Controls match these attributes while changing component provenance. Cross-layout consistency is evaluated in offset space.

10.2 Adaptive considerations

A knowledgeable attacker may move or split malicious functionality, interleave it with benign code, or add decoy high-scoring regions. We evaluate bounded transformations and discuss the remaining adaptive surface. The system is an analyst-prioritization aid, not a standalone guarantee that unselected code is benign.

10.3 Parser and disassembler disagreement

We compare at least two PE parsers and, on a subset, at least two disassembly backends. Disagreement is recorded as annotation uncertainty. Samples with unresolved mapping differences are excluded from primary gold metrics under a preregistered rule and retained for failure analysis.

11. Case Studies

Each case study will include:

1. a PEAtlas view with ranked intervals;
2. original file offsets and mapped RVAs;
3. functions/basic blocks in the reverse-engineering tool;
4. source or analyst evidence supporting the label;
5. comparison with capa, DeepReflect, and one MIL baseline;
6. counterfactual deletion and layout-stability checks;
7. analysis of false positives and missed components.

At least one success, one partial success, and one failure case will be shown. Case studies will not replace aggregate evaluation.

12. Discussion

12.1 What localization means

UA-SAHI-Mal retrieves regions associated with a file-level decision and validated against available evidence. It does not prove causality in the program-execution sense. A component can be security-relevant because it enables persistence, evasion, command and control, credential access, or another malicious objective, but static evidence may be incomplete or ambiguous.

12.2 Why not train a detector directly?

We do train supervised baselines on available gold data. The weak formulation is motivated by the scale mismatch between file labels and component annotations, not by a claim that supervised detection is conceptually invalid. The comparison quantifies when weak supervision is useful and where it falls short.

12.3 Why use an image representation?

Images provide a convenient multiscale spatial substrate and access to mature visual encoders and upsamplers. However, the one-dimensional byte domain remains authoritative. The representation comparison and cross-layout objective test whether any benefit survives arbitrary rasterization choices.

12.4 Relationship to SAHI

The proposed selection mechanism is SAHI-inspired. We reserve the term "Full SAHI" for experiments that run an actual detector over all slices and merge detections according to the SAHI protocol. Other experiments are described as hierarchical slicing to avoid conflating patch MIL with object detection.

13. Ethics, Safety, and Artifact Handling

The project handles malware only in isolated, access-controlled environments. Samples are never executed on production systems. Raw malware redistribution follows source licenses, provider agreements, institutional policy, and applicable law. Where binaries cannot be redistributed, the artifact releases hashes, build recipes, derived annotations, feature records, and evaluation code to the extent permitted.

Controlled-provenance modules are designed to avoid harmful external effects. Analyst-study data are pseudonymized, and participants receive informed consent and compensation under applicable institutional review. The system is framed as decision support; analysts are warned that unselected regions may still contain malicious behavior.

14. Limitations

1. Static analysis can miss runtime-decrypted, dynamically resolved, self-modifying, or environment-dependent behavior.
2. Source-grounded builds may differ from in-the-wild malware in optimization, packing, compiler, and operational context.
3. YARA and capa provide partial, rule-dependent evidence and cannot serve as complete maliciousness ground truth.
4. Debug information can describe code layout without deciding whether a function is malicious; analyst adjudication remains necessary.
5. File-level labels may not identify distributed multi-function behavior, even with MIL and counterfactual constraints.
6. UA may sharpen structural boundaries that are unrelated to semantic behavior and may not justify its online cost.
7. The primary gold corpus may remain small relative to file-level detection datasets, limiting family coverage.
8. An adaptive attacker can target the selector with decoys or distributed payloads.
9. Results for unpacked native PE files may not generalize to packed files, .NET, drivers, scripts, or other formats.

15. Related Work

Static PE detection and malware images

Malware-image classification maps byte patterns to visual texture (Nataraj et al. 2011). Raw-byte networks such as MalConv avoid image transformation (Raff et al. 2018), while EMBER provides a strong engineered-feature baseline (Anderson and Roth 2018). EMBER2024 expands formats, tasks, and temporal evaluation (Joyce et al. 2025). UA-SAHI-Mal differs by targeting independently evaluated component evidence under a budget.

Malware component localization and explainability

DeepReflect localizes anomalous functions and incrementally clusters malware functionality (Downing et al. 2021). capa matches expert-defined capabilities (Mandiant FLARE Team 2026), and DLLicious uses position-preserving attribution to indicate byte regions associated with malicious DLL patterns (Yevsikov and Nissim 2026). These are direct baselines or complementary evidence sources.

Weak supervision and MIL

ABMIL introduced attention-based permutation-invariant aggregation (Ilse, Tomczak, and Welling 2018). CLAM, DSMIL, and TransMIL model representative and correlated regions in very high-resolution weakly labeled images (Lu et al. 2021; Li, Li, and Eliceiri 2021; Shao et al. 2021). Peters and Farhat applied patch MIL to high-resolution malware images (Peters and Farhat 2023), and PE-MILCon combines byte-segment MIL with PE structural features (Kim, Trung, and Pham 2026). Our focus is not attention visualization alone, but budgeted retrieval evaluated against external evidence.

High-resolution selective inference

SAHI performs detector inference over slices to recover small objects (Akyon, Altinuc, and Temizel 2022). Upsample Anything reconstructs low-resolution features and probability maps using per-input, edge-aware optimization (Seo, Hamilton, and Kim 2026). UA-SAHI-Mal adapts these concepts to a weakly supervised byte-evidence task and evaluates whether their added cost and representation assumptions are justified.

16. Conclusion

This paper defines static PE malware localization as budgeted retrieval of analyst-verifiable byte evidence rather than direct prediction of unverified image boxes. UA-SAHI-Mal combines an invertible PE representation, weak tile learning, structure-guided reconstruction, guarded subset selection, and recursive refinement. Its evaluation separates controlled provenance, silver rules, source-grounded gold, and analyst utility. The design deliberately avoids treating model attention, injected markers, or capability rules as interchangeable ground truth. The empirical study will determine whether the proposed system improves the evidence-recall–cost frontier and reduces reverse-engineering workload. Until those experiments are complete, all performance claims remain open hypotheses.

Appendix A. Planned Systematic Audit Protocol

The final appendix will include:

- databases and repositories searched;
- exact search strings;
- search and update dates;
- language and publication filters;
- inclusion criteria;
- duplicate handling;

- two-reviewer screening procedure;
- evidence extraction form;
- reasons each dataset fails or satisfies the five benchmark conditions;
- a public machine-readable audit table.

Appendix B. Annotation Schema

```
{
  "sample_sha256": "...",
  "build_id": "...",
  "coordinate_version": "peatlas-v1",
  "component_id": "...",
  "label_class": "MALICIOUS_COMPONENT",
  "semantic_tags": ["..."],
  "file_offset_intervals": [[1000, 1040], [2048, 2112]],
  "rva_intervals": [[8192, 8232], [12288, 12352]],
  "mapped_functions": ["sub_401000"],
  "source_locations": ["module.c:20-71"],
  "provenance": "SOURCE_AND_ANALYST",
  "annotator_agreement": "AGREED",
  "notes": "..."
}
```

Appendix C. Minimum Reproducibility Checklist

- source code and pinned environments;
- dataset and split hashes;
- PEAtlas mapping tests;
- compiler/build containers for controlled and gold builds;
- model checkpoints and hyperparameters;
- all baseline adapters;
- inference-time accounting script;
- annotation guidelines and adjudication log;
- statistical-analysis notebook;
- failure-case inventory;
- malware-handling and access instructions.

Akyon, Fatih Cagatay, Sinan Onur Altinuc, and Alptekin Temizel. 2022. "Slicing Aided Hyper Inference and Fine-Tuning for Small Object Detection." In *2022 IEEE International Conference on Image Processing (ICIP)*, 966–70. https://doi.org/10.1109/ICIP46576.2022.9897990.

Anderson, Hyrum S., and Phil Roth. 2018. "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models." *arXiv Preprint arXiv:1804.04637*. https://arxiv.org/abs/1804.04637.

Demetrio, Luca, and Battista Biggio. 2021. "Secml-Malware: A Python Library for Adversarial Robustness Evaluation of Windows Malware Classifiers." *arXiv Preprint arXiv:2104.12848*. https://github.com/pralab/secml_malware.

Downing, Evan, Yisroel Mirsky, Kyuhong Park, and Wenke Lee. 2021. "DeepReflect: Discovering Malicious Functionality Through Binary Reconstruction." In *30th USENIX Security Symposium (USENIX Security 21)*, 3469–86. USENIX Association. https://www.usenix.org/conference/usenixsecurity21/presentation/downing.

Ilse, Maximilian, Jakub Tomczak, and Max Welling. 2018. "Attention-Based Deep Multiple Instance Learning." In *Proceedings of the 35th International Conference on Machine Learning*, 80:2127–36. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v80/ilse18a.html.

Joyce, Robert J., and collaborators. 2026. "EMBER2024-capa: Function-Level Capability, Byte, and

Disassembly Records." Hugging Face dataset. https://huggingface.co/datasets/joyce8/EMBER2024.

Joyce, Robert J., Gideon Miller, Phil Roth, Richard Zak, Elliott Zaresky-Williams, Hyrum Anderson, Edward Raff, and James Holt. 2025. "EMBER2024: A Benchmark Dataset for Holistic Evaluation of Malware Classifiers." In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/3711896.3737431.

Kim, Tuan Nguyen, Son Doan Trung, and Nguyen Minh Nhut Pham. 2026. "PE-MILCon: Multiple-Instance Learning with Contrastive Multi-View Representation for Static Windows PE Malware Detection." *Computers, Materials & Continua* 89 (1): 43. https://doi.org/10.32604/cmc.2026.084268.

Li, Bin, Yin Li, and Kevin W. Eliceiri. 2021. "Dual-Stream Multiple Instance Learning Network for Whole Slide Image Classification with Self-Supervised Contrastive Learning." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. https://arxiv.org/abs/2011.08939.

Lu, Ming Y., Drew F. K. Williamson, Tiffany Y. Chen, Richard J. Chen, Matteo Barbieri, and Faisal Mahmood. 2021. "Data-Efficient and Weakly Supervised Computational Pathology on Whole-Slide Images." *Nature Biomedical Engineering* 5: 555–70. https://doi.org/10.1038/s41551-020-00682-w.

Mandiant FLARE Team. 2026. "Capa: Extract Capabilities from Executable Files." Software and documentation. https://mandiant.github.io/capa/.

Microsoft. 2021. "Debugging Optimized Code and Inline Functions." https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/debugging-optimized-code-and-inline-functions-external.

———. 2025a. "/OPT (Optimizations)." https://learn.microsoft.com/en-us/cpp/build/reference/opt-optimizations.

———. 2025b. "PE Format." https://learn.microsoft.com/en-us/windows/win32/debug/pe-format.

Nataraj, Lakshmanan, Sreejith Karthikeyan, Gregoire Jacob, and B. S. Manjunath. 2011. "Malware Images: Visualization and Automatic Classification." In *Proceedings of the 8th International Symposium on Visualization for Cyber Security*. https://doi.org/10.1145/2016904.2016908.

Peters, Tim, and Hikmat Farhat. 2023. "High-Resolution Image-Based Malware Classification Using Multiple Instance Learning." *arXiv Preprint arXiv:2311.12760*. https://doi.org/10.48550/arXiv.2311.12760.

Raff, Edward, Jon Barker, Jared Sylvester, Robert Brandon, Bryan Catanzaro, and Charles Nicholas. 2018. "Malware Detection by Eating a Whole EXE." *arXiv Preprint arXiv:1710.09435*. https://arxiv.org/abs/1710.09435.

Seo, Minseok, Mark Hamilton, and Changick Kim. 2026. "Upsample Anything: A Simple and Hard to Beat Baseline for Feature Upsampling." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. https://arxiv.org/abs/2511.16301.

Shao, Zhuchen, Hao Bian, Yang Chen, Yifeng Wang, Jian Zhang, Xiangyang Ji, and Yongbing Zhang. 2021. "TransMIL: Transformer-Based Correlated Multiple Instance Learning for Whole Slide Image Classification." In *Advances in Neural Information Processing Systems*. Vol. 34. https://proceedings.neurips.cc/paper/2021/hash/10c272d06794d3e5785d5e7c5356e9ff-Abstract.html.

VirusTotal. 2026. "YARA c API Documentation." https://yara.readthedocs.io/en/latest/capi.html.

Yang, Limin, Arridhana Ciptadi, Ihar Laziuk, Ali Ahmadzadeh, and Gang Wang. 2021. "BODMAS: An Open Dataset for Learning-Based Temporal Analysis of PE Malware." In *4th Deep Learning and Security Workshop*. https://whyisyoung.github.io/BODMAS/.

Yevsikov, Alexander, and Nir Nissim. 2026. "DLLicious: Detection and Explainability of Malicious DLL Files Using Novel Static Feature Extraction Methods." *Expert Systems with Applications* 320: 132188. https://doi.org/10.1016/j.eswa.2026.132188.